

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ

---□□4---



BÁO CÁO CUỐI KỲ

Môn học: Nhập môn trí tuệ nhân tạo

ĐỀ TÀI: *Nghiên cứu và xây dựng hệ thống trí tuệ nhân tạo cho trò chơi cờ vua*

Giảng viên: GS.TS Nguyễn Thế Toàn
Thầy Vi Anh Quân

Nhóm: 4

Tên thành viên:	<i>Nguyễn Thị Hà</i>	<i>23001602</i>
	<i>Phạm Minh Trí</i>	
	<i>Nguyễn Tuấn Đạt</i>	<i>23001594</i>
	<i>Trần Danh Đoàn</i>	<i>23001597</i>

Hà Nội 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài, nhóm chúng em đã nhận được sự quan tâm, hướng dẫn và giúp đỡ quý báu từ các thầy và bạn bè xung quanh.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc tới các thầy trong Bộ môn Tin học vật lý nói riêng và các thầy cô của Trường nói chung đã tận tình giảng dạy, truyền đạt những kiến thức nền tảng quý báu trong suốt quá trình học tập. Những kiến thức đó là cơ sở quan trọng giúp chúng em hoàn thành đề tài này.

Đặc biệt, chúng em xin gửi lời cảm ơn chân thành tới thầy Vi Anh Quân đã tận tình chỉ bảo, định hướng và đưa ra nhiều ý kiến đóng góp quý báu trong suốt quá trình thực hiện đề tài. Sự hướng dẫn tận tâm của thầy đã giúp chúng em từng bước hoàn thiện nội dung nghiên cứu và nâng cao khả năng vận dụng kiến thức vào thực tế.

Mặc dù đã có nhiều cố gắng trong quá trình thực hiện, song do thời gian và kiến thức còn hạn chế nên báo cáo khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp quý báu từ thầy để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Hà Nội, ngày 11 tháng 06 năm 2026

PHÂN CHIA CÔNG VIỆC

Thành viên	Vai trò	Đóng góp
Nguyễn Thị Hà	Viết code	25%
Phạm Minh Trí	Hỗ trợ kiểm thử code và thuyết trình	25%
Nguyễn Tuấn Đạt	Tổng hợp nội dung, Làm báo cáo	25%
Trần Danh Đoàn	Tổng hợp nội dung, làm slide	25%

LỜI MỞ ĐẦU.....	6
CHƯƠNG I: TỔNG QUAN VỀ ĐỀ TÀI.....	1
1.1.Giới thiệu.....	1
1.2.Mục tiêu của đề tài.....	2
1.3.Phạm vi nghiên cứu.....	3
1.3.1. Nội dung nghiên cứu	3
1.3.2. Giới hạn nghiên cứu	3
1.3.3. Đối tượng nghiên cứu.....	4
1.3.4. Kết quả dự kiến	4
2.4. Tìm kiếm Quiescence (Quiescence Search).....	13
2.4.1. Hiệu ứng chân trời (Horizon Effect)	13
2.4.2. Giải pháp Quiescence Search	13
2.4.3 Cài đặt trong dự án.....	14
2.5. Hàm đánh giá tĩnh bàn cờ.....	15
2.5.1. Giá trị vật chất (Material Value).....	15
2.5.2.Piece-Square Tables	15
2.5.3. Các thành phần đánh giá bổ sung.....	16
2.6. Sắp xếp nước đi (Move Ordering)	17
2.6.1. Nguyên lý.....	17
2.6.2. Hàm đánh giá thứ tự nước đi	17
2.6.3. Vai trò trong dự án	18
CHƯƠNG III: THIẾT KẾ HỆ THỐNG	19
3.1.Kiến trúc hệ thống.....	19
3.2.Luồng xử lý chính.....	19
3.3.Module GameController (main.py)	20
3.3.1.Khởi tạo và cấu hình	20
3.3.2.Xử lý đa luồng.....	20
3.3.3.Kiểm tra trạng thái kết thúc	21
3.4.Module giao diện (GUI.py)	21
3.5.Tích hợp thư viện chess (python-chess).....	21
CHƯƠNG IV.THUẬT TOÁN AI - PHÂN TÍCH CHI TIẾT	23
4.1.Cấu Trúc Hàm find_best_move()	23
4.2.Phân Tích Hàm evaluate_board()	23

4.2.1 Xử Lý Trạng Thái Kết Thúc	23
4.2.2 Xử Lý Đúng Hướng PST Cho Hai Màu Quân	23
4.2.3 Phát Hiện Giai Đoạn Tàn Cuộc	24
4.3.Chi Tiết Triển Khai Quiescence Search.....	24
4.4.Phân Tích Độ Phức Tạp và Hiệu Suất	25
CHƯƠNG V. GIAO DIỆN NGƯỜI DÙNG	25
5.1.Thiết Kế Giao Diện.....	25
5.2.Màn Hình Chọn Phe (Start Menu).....	26
5.3.Màn Hình Chơi Chính.....	26
5.5.Popup Kết Quả.....	27
CHƯƠNG VI. KIỂM THỬ VÀ ĐÁNH GIÁ	28
6.1.Phương Pháp Kiểm Thử	28
6.1.1.Kiểm Thử Luật Cờ Vua.....	28
6.1.2.Kiểm Thử Hàm Đánh Giá	29
6.1.3.Đánh Giá Chất Lượng AI	29
6.3.Ước Tính ELO	29
CHƯƠNG VII. KẾT LUẬN, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN.....	30
7.1.Kết luận	30
7.2.Hạn chế.....	31
7.2.1 Hạn Chế Kỹ Thuật AI	31
7.2.1.Hạn Chế Giao Diện	31
7.3.Hướng Phát Triển Tương Lai	32
7.3.1 Cải Tiến Engine AI	32
7.3.1.Mở Rộng Tính Năng	32
TÀI LIỆU THAM KHẢO.....	33

DANH MỤC HÌNH ẢNH

<i>Hình 1.1: Bàn cờ vua tiêu chuẩn.....</i>	<i>2</i>
<i>Hình 2.2: Minh họa thuật toán minimax.....</i>	<i>9</i>
<i>Hình 2.3 Cơ chế sắp xếp nước đi Move Ordering trong hàm alpha_beta().....</i>	<i>10</i>
<i>Hình 2.4: Minh họa cắt tỉa Alpha-Beta - beta-cutoff tại Nhánh B.....</i>	<i>12</i>
<i>Hình 2.5: Cài đặt cắt tỉa Alpha-Beta trong minimax.py.....</i>	<i>13</i>
<i>Hình 2.6: Quiescence Search ngăn đánh giá sai tại nút lá.....</i>	<i>14</i>
<i>Hình 2.7: Cài đặt hàm quiescence_search() trong minimax.py.....</i>	<i>15</i>

Hình 2.8. Piece-Square Table áp dụng cho quân Tốt trong minimax.py.....	16
Hình 2.9 Phát hiện giai đoạn tàn cuộc (Endgame Detection).....	17
Hình 2.10. Sắp xếp danh sách nước đi theo giá trị score_move().....	18
Hình 5.1. Giao diện chính.....	26
Hình 5.2. Màn hình chơi chính.....	27
Hình 5.3. Kết quả ván đấu.....	28

LỜI MỞ ĐẦU

Trong những năm gần đây, trí tuệ nhân tạo (Artificial Intelligence - AI) đã trở thành một trong những lĩnh vực nghiên cứu và phát triển quan trọng của khoa học máy tính. AI không chỉ được ứng dụng trong các hệ thống nhận dạng hình ảnh, xử lý ngôn ngữ tự nhiên hay xe tự hành mà còn đóng vai trò quan trọng trong các trò chơi mang tính chiến lược. Trong đó, cờ vua được xem là một bài toán kinh điển để nghiên cứu các thuật toán tìm kiếm và ra quyết định trong trí tuệ nhân tạo.

Cờ vua là trò chơi đối kháng giữa hai người chơi với số lượng trạng thái và nước đi rất lớn. Để xây dựng một hệ thống có khả năng chơi cờ hiệu quả, máy tính cần phải đánh giá được các trạng thái bàn cờ và lựa chọn nước đi tối ưu trong nhiều khả năng khác nhau. Đây là bài toán tiêu biểu cho việc áp dụng các thuật toán tìm kiếm trong không gian trạng thái.

Thuật toán Minimax là một trong những phương pháp cơ bản và phổ biến nhất được sử dụng để giải quyết các trò chơi đối kháng hai người chơi. Tuy nhiên, khi độ sâu tìm kiếm tăng lên, số lượng trạng thái cần xem xét tăng rất nhanh, dẫn đến chi phí tính toán lớn. Để khắc phục hạn chế này, kỹ thuật Alpha-Beta Pruning được sử dụng nhằm loại bỏ các nhánh không cần thiết trong cây tìm kiếm, từ đó giảm thời gian xử lý mà vẫn đảm bảo kết quả tối ưu.

Xuất phát từ những lý do trên, đề tài “Nghiên cứu và xây dựng hệ thống trí tuệ nhân tạo chơi cờ vua sử dụng thuật toán Minimax và Alpha-Beta Pruning” được thực hiện nhằm tìm hiểu nguyên lý hoạt động của các thuật toán tìm kiếm trong AI, đồng thời xây dựng một chương trình có khả năng chơi cờ vua tự động. Thông qua việc triển khai và đánh giá thực nghiệm, đề tài giúp làm rõ hiệu quả của kỹ thuật Alpha-Beta Pruning trong việc tối ưu hóa thuật toán Minimax, góp phần nâng cao hiệu suất của hệ thống AI chơi cờ vua.

Nội dung báo cáo gồm các phần chính: tổng quan về trí tuệ nhân tạo và cờ vua, cơ sở lý thuyết của thuật toán Minimax và Alpha-Beta Pruning, thiết kế và xây dựng hệ thống, thực nghiệm đánh giá kết quả, cuối cùng là các kết luận và hướng phát triển trong tương lai.

CHƯƠNG I: TỔNG QUAN VỀ ĐỀ TÀI

1.1. Giới thiệu

Cờ vua, được biết đến với tên gọi tiếng Anh là "chess," là một trò chơi đối kháng trí tuệ chơi trên bảng dành cho hai người. Xuất phát từ những trò chơi cổ có nguồn gốc từ Ấn Độ và Ba Tư, cờ vua đã trải qua một quá trình phát triển dài lâu để trở thành hình thức chơi hiện đại ngày nay. Đặc biệt, vào nửa sau của thế kỷ 15, cờ vua đã chầm ngòi cho sự lan rộng và phổ biến ở Nam Âu.



Hình 1.1: Bàn cờ vua tiêu chuẩn

Trò chơi diễn ra trên một bàn cờ vuông chia thành 64 ô, với mỗi người chơi điều khiển một quân đội bao gồm tốt, mã, tượng, xe, hậu và vua. Mục tiêu chính của người chơi là chiếu hết vua đối phương, nơi vua bị đe dọa mà không có cách nào bảo vệ.

Trong thế kỷ 20 và đặc biệt là thế kỷ 21, sự phát triển của trí tuệ nhân tạo đã đánh dấu một cột mốc quan trọng trong lịch sử cờ vua, biến những bảng cờ truyền thống thành cuộc đấu giữa con người và máy tính. Một số sự kiện đáng chú ý đã chứng kiến sức mạnh đáng kinh ngạc của máy tính trong trò chơi này.

Năm 1997, Deep Blue của IBM đánh bại Garry Kasparov, đánh dấu cột mốc đầu tiên máy tính vượt qua nhà vô địch thế giới trong một trận đấu chính thức. Sau đó, các engine hiện đại như Stockfish tiếp tục nâng cao sức mạnh nhờ sự kết hợp giữa các thuật

toán tìm kiếm tối ưu, hàm đánh giá mạnh và nhiều kỹ thuật tối ưu hóa chuyên sâu. Năm 2017, AlphaZero của DeepMind đã mở ra hướng tiếp cận mới khi sử dụng học tăng cường (Reinforcement Learning) để tự học và đạt trình độ siêu cao mà không cần dữ liệu khai cuộc do con người cung cấp.

Chính vì tính đối kháng, số lượng trạng thái khổng lồ và yêu cầu ra quyết định liên tục, cờ vua trở thành môi trường lý tưởng để nghiên cứu các thuật toán tìm kiếm như Minimax và Alpha – Beta Pruning cũng như đánh giá hiệu quả của các hệ thống trí tuệ nhân tạo.

1.2.Mục tiêu của đề tài

Đề tài được thực hiện nhằm nghiên cứu và ứng dụng các thuật toán tìm kiếm trong lĩnh vực trí tuệ nhân tạo để xây dựng một hệ thống có khả năng chơi cờ vua tự động.

Thông qua việc triển khai thuật toán Minimax kết hợp với kỹ thuật Alpha–Beta Pruning, đề tài hướng đến việc mô phỏng quá trình ra quyết định của máy tính trong môi trường đối kháng, đồng thời đánh giá hiệu quả của các phương pháp tối ưu hóa tìm kiếm đối với chất lượng nước đi và hiệu năng của hệ thống

Để đạt được mục tiêu trên đề tài tập trung thực hiện các nội dung sau:

Xây dựng chương trình AI có khả năng chơi cờ vua dựa trên thuật toán Minimax.

Tích hợp kỹ thuật Alpha–Beta Pruning nhằm giảm số lượng nút phải duyệt và nâng cao hiệu suất tìm kiếm.

Thiết kế hàm đánh giá (Evaluation Function) để ước lượng mức độ lợi thế của các trạng thái bàn cờ và hỗ trợ quá trình ra quyết định của AI.

Tích hợp cơ chế sinh nước đi hợp lệ và quản lý trạng thái bàn cờ thông qua thư viện python-chess.

Thực hiện thực nghiệm với nhiều độ sâu tìm kiếm khác nhau để đánh giá hiệu năng của hệ thống thông qua các chỉ số như thời gian thực thi, số lượng nút được duyệt và chất lượng nước đi được lựa chọn.

Phân tích và so sánh hiệu quả giữa thuật toán Minimax thuần túy và Minimax kết hợp Alpha–Beta Pruning nhằm làm rõ vai trò của kỹ thuật cắt tỉa trong việc tối ưu hóa quá trình tìm kiếm.

1.3. Phạm vi nghiên cứu

1.3.1. Nội dung nghiên cứu

Đề tài tập trung nghiên cứu và triển khai các nội dung chính sau:

- Tìm hiểu cơ sở lý thuyết về trí tuệ nhân tạo trong các trò chơi đối kháng và vai trò của các thuật toán tìm kiếm trong quá trình ra quyết định.
- Phân tích nguyên lý hoạt động của thuật toán Minimax và ứng dụng thuật toán trong việc lựa chọn nước đi tối ưu cho hệ thống AI.
- Nghiên cứu kỹ thuật Alpha–Beta Pruning nhằm tối ưu hóa quá trình tìm kiếm bằng cách loại bỏ các nhánh không cần thiết trong cây trò chơi.
- Thiết kế và xây dựng hàm đánh giá (Evaluation Function) để lượng hóa mức độ lợi thế của từng trạng thái bàn cờ dựa trên giá trị quân cờ và một số yếu tố chiến thuật cơ bản.
- Thiết kế và triển khai chương trình AI chơi cờ vua có khả năng sinh nước đi hợp lệ, đánh giá trạng thái bàn cờ và lựa chọn nước đi tối ưu dựa trên thuật toán Minimax kết hợp với kỹ thuật Alpha–Beta Pruning.

1.3.2. Giới hạn nghiên cứu

Đề tài được thực hiện trong phạm vi nghiên cứu sau:

- Hệ thống chỉ áp dụng cho trò chơi cờ vua tiêu chuẩn trên bàn cờ kích thước 8×8
- AI được xây dựng dựa trên thuật toán Minimax kết hợp với kỹ thuật Alpha–Beta Pruning và không sử dụng các phương pháp Machine Learning, Deep Learning hoặc Reinforcement Learning.
- Độ sâu tìm kiếm được giới hạn ở các mức phù hợp nhằm cân bằng giữa chất lượng nước đi và thời gian phản hồi của hệ thống, đồng thời đáp ứng khả năng tính toán của máy tính phổ thông.
- Hàm đánh giá chủ yếu dựa trên giá trị vật chất của quân cờ kết hợp với một số yếu tố vị trí cơ bản, chưa xét đầy đủ các chiến thuật và chiến lược chuyên sâu trong khai cuộc, trung cuộc và tàn cuộc.
- Hệ thống không hướng đến việc cạnh tranh với các chess engine chuyên nghiệp như Stockfish hoặc Leela Chess Zero mà tập trung vào việc nghiên cứu nguyên

lý hoạt động của các thuật toán tìm kiếm và đánh giá hiệu quả khi áp dụng vào bài toán cờ vua.

1.3.3. Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là các thuật toán tìm kiếm trong lĩnh vực trí tuệ nhân tạo, trọng tâm là thuật toán Minimax và kỹ thuật Alpha–Beta Pruning cùng việc ứng dụng chúng trong bài toán xây dựng hệ thống AI chơi cờ vua.

1.3.4. Kết quả dự kiến

Sau khi hoàn thành, đề tài hướng đến việc xây dựng thành công một chương trình AI chơi cờ vua có khả năng:

- Tích hợp cơ chế sinh nước đi hợp lệ và quản lý trạng thái bàn cờ thông qua thư viện python-chess.
- Đánh giá trạng thái bàn cờ thông qua hàm đánh giá được thiết kế.
- Lựa chọn nước đi tối ưu bằng thuật toán Minimax.
- Nâng cao hiệu quả tìm kiếm thông qua việc áp dụng kỹ thuật Alpha–Beta Pruning.
- Thi đấu với người chơi ở mức độ cơ bản đến trung bình, đồng thời thể hiện khả năng lựa chọn nước đi hợp lý trong các tình huống thường gặp.

CHƯƠNG II: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về trò chơi cờ vua và bài toán AI

2.1.1. Phân loại bài toán

Trong lĩnh vực trí tuệ nhân tạo, cờ vua được xem là một bài toán tìm kiếm đối kháng (Adversarial Search Problem) với các đặc điểm chính sau:

- Thông tin hoàn toàn (Perfect Information): Cả hai người chơi đều quan sát được toàn bộ trạng thái bàn cờ tại mọi thời điểm. Không tồn tại thông tin ẩn như trong các trò chơi bài Poker hoặc Battleship.

- **Tất định (Deterministic):** Kết quả của mỗi nước đi được xác định hoàn toàn bởi trạng thái hiện tại và hành động của người chơi, không tồn tại yếu tố ngẫu nhiên như việc tung xúc xắc trong một số trò chơi khác.
- **Game đối kháng (Two – player adversarial):** Hai người chơi có mục tiêu hoàn toàn đối lập nhau. Mỗi nước đi của một bên đều hướng tới việc tối đa hóa lợi thế của mình đồng thời làm giảm lợi thế của đối phương.
- **Tổng bằng không (Zero-sum):** Tổng giá trị lợi ích của hai người chơi được xem là không đổi. Mọi lợi thế mà một bên đạt được sẽ tương ứng với sự bất lợi của bên kia.
- **Hữu hạn (Finite):** Mặc dù số lượng trạng thái và nước đi bàn cờ là cực kỳ lớn, không gian trạng thái vẫn là hữu hạn do chịu ràng buộc bởi luật chơi (chẳng hạn luật 50 nước và quy tắc lặp lại thế cờ)

Những đặc điểm trên khiến cờ vua trở thành một trong những bài toán kinh điển để nghiên cứu và ứng dụng các thuật toán tìm kiếm đối kháng, đặc biệt là thuật toán Minimax và kỹ thuật Alpha–Beta Pruning.

2.1.2. Cây trò chơi (Game Tree)

Trong trí tuệ nhân tạo, quá trình ra quyết định trong cờ vua thường được mô hình hóa dưới dạng cây trò chơi (Game Tree). Mỗi trạng thái và nước đi được biểu diễn như sau:

- Mỗi nút (Node) biểu diễn một trạng thái bàn cờ, bao gồm vị trí các quân cờ, quyền nhập thành, trạng thái bắt tốt qua đường và bên được quyền đi tiếp.
- Mỗi cạnh (Edge) đại diện cho một nước đi hợp lệ chuyển từ trạng thái hiện tại sang trạng thái kế tiếp.
- Các nút lá (Leaf Nodes) có thể là trạng thái kết thúc của ván cờ (chiếu hết, hòa cờ) hoặc các trạng thái đạt đến giới hạn độ sâu tìm kiếm đã thiết lập.
- Hệ số nhánh trung bình (average branching factor) $b \approx 35$ đối với cờ vua, nghĩa là từ mỗi trạng thái bàn cờ thường tồn tại khoảng 35 nước đi hợp lệ.

Mục tiêu của AI là tại mỗi lượt đi sẽ duyệt cây trò chơi đến một độ sâu xác định, sau đó lựa chọn nước đi mang lại giá trị đánh giá tối ưu dựa trên giả định rằng đối thủ cũng luôn thực hiện các nước đi tối ưu.

2.1.3. Độ phức tạp và sự cần thiết của tối ưu hóa

Thuật toán Minimax thuần túy có độ phức tạp thời gian ($O(b^d)$), trong đó: (b) là hệ số phân nhánh trung bình (Branching Factor) và (d) là độ sâu tìm kiếm.

Đối với cờ vua, giá trị trung bình của (b) vào khoảng 35, khiến số lượng trạng thái cần duyệt tăng theo cấp số nhân khi tăng độ sâu tìm kiếm.

Ví dụ, với độ sâu tìm kiếm ($d = 4$), thuật toán Minimax cần đánh giá khoảng $(35^4 = 1.500.625)$ nút. Khi tăng lên ($d = 6$), số lượng nút cần duyệt lên tới $(35^6 \approx 1.8)$ tỷ nút, vượt quá khả năng xử lý trong thời gian ngắn trên các máy tính thông thường.

Do đó, việc áp dụng các kỹ thuật tối ưu như Alpha–Beta Pruning, Move Ordering và Quiescence Search là cần thiết nhằm giảm số lượng nút phải đánh giá, từ đó rút ngắn thời gian tìm kiếm nhưng vẫn đảm bảo chất lượng của nước đi được lựa chọn.

Bảng 2.1 trình bày số lượng nút cần duyệt của thuật toán Minimax thuần túy và trường hợp Alpha–Beta Pruning đạt hiệu quả lý tưởng ở một số độ sâu tìm kiếm tiêu biểu.

Độ sâu (d)	Minimax thuần túy (nút)	Alpha – Beta (lý tưởng)	Tỷ lệ giảm
2	1,225	70	≈ 17 lần
4	1,500,625	$\approx 2,450$	≈ 612 lần
6	1,838,265,625	$\approx 85,750$	$\approx 21,438$ lần

Bảng 2.1. So sánh số nút duyệt giữa Minimax và Alpha-Beta

Từ bảng trên có thể thấy rằng khi độ sâu tìm kiếm tăng, số lượng nút của thuật toán Minimax tăng rất nhanh theo hàm mũ. Trong khi đó, việc kết hợp kỹ thuật Alpha–Beta Pruning giúp cắt bỏ nhiều nhánh không cần thiết của cây trò chơi, làm giảm đáng kể chi phí tính toán và cho phép hệ thống tìm kiếm ở độ sâu lớn hơn trong cùng khoảng thời gian xử lý.

2.2 Thuật toán Minimax

2.2.1. Nguyên lý hoạt động

Thuật toán Minimax là một trong những thuật toán tìm kiếm đối kháng (Adversarial Search) kinh điển trong lĩnh vực trí tuệ nhân tạo. Thuật toán được xây dựng trên giả định rằng cả hai người chơi đều luôn đưa ra những quyết định tối ưu nhằm tối đa hóa lợi ích của mình và giảm thiểu lợi thế của đối phương.

Trong cây trò chơi, hai người chơi được mô hình hóa thành hai tác nhân có mục tiêu trái ngược nhau:

Thành phần	Vai trò
Max (Bot)	Lựa chọn nước đi làm tối đa hóa giá trị đánh giá của bàn cờ
Min (Player)	Lựa chọn nước đi làm tối thiểu hóa giá trị đánh giá của bàn cờ
Nút lá (Leaf Node)	Giá trị được tính qua hàm đánh giá tĩnh <code>evaluate_board()</code>
Nút trung gian (Internal Node)	Giá trị được tính từ max/min của các nút con

Với cơ chế này, thuật toán Minimax sẽ duyệt cây trò chơi đến một độ sâu xác định và lan truyền giá trị từ các nút lá lên nút gốc để lựa chọn nước đi tối ưu cho AI

Trong dự án này, thuật toán Minimax không được triển khai dưới dạng thuần túy mà được mở rộng bằng kỹ thuật cắt tỉa Alpha-Beta, sắp xếp thứ tự nước đi (Move Ordering), mở rộng khi đang bị chiếu (Check Extension) và Quiescence Search nhằm nâng cao hiệu quả tìm kiếm cũng như chất lượng nước đi của hệ thống A

2.2.2. Minh họa bằng ví dụ

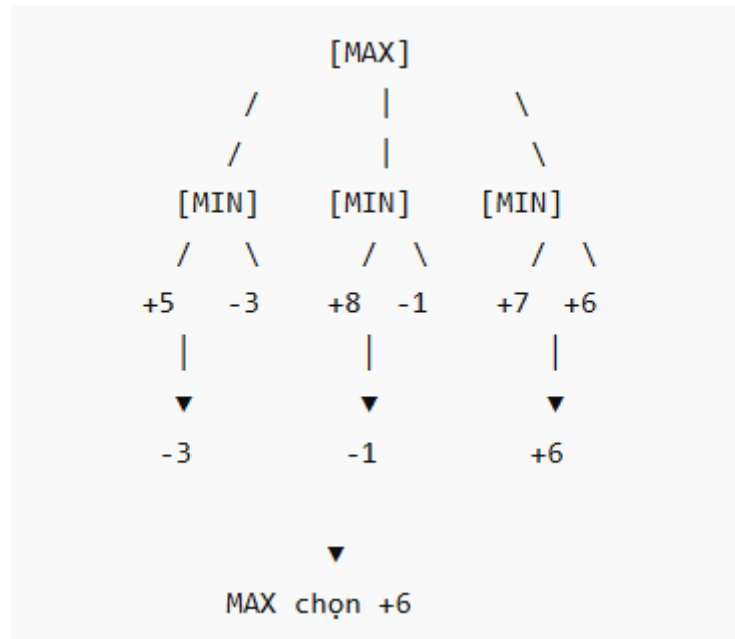
Xét một cây tìm kiếm có độ sâu 2, trong đó tầng đầu tiên là lượt đi của BOT (MAX) và tầng tiếp theo là lượt đi của người chơi (MIN). Mỗi trạng thái giả sử có ba nước đi hợp lệ.

Các nút lá được gán giá trị bởi hàm `evaluate_board()`, trong đó giá trị dương biểu thị trạng thái có lợi cho BOT và giá trị âm biểu thị trạng thái có lợi cho người chơi.

Dựa trên nguyên lý của thuật toán Minimax:

- Tại **nhánh A**, người chơi (MIN) sẽ lựa chọn giá trị nhỏ nhất là **-3**.
- Tại **nhánh B**, người chơi (MIN) sẽ lựa chọn giá trị nhỏ nhất là **-1**.
- Tại **nhánh C**, người chơi (MIN) sẽ lựa chọn giá trị nhỏ nhất là **6**.

Sau khi các giá trị được lan truyền ngược lên nút gốc, BOT (MAX) sẽ lựa chọn giá trị lớn nhất trong ba nhánh, tức là **6**, và thực hiện nước đi tương ứng.



Hình 2.2: Minh họa thuật toán minimax

Ví dụ trên cho thấy thuật toán Minimax luôn giả định đối thủ cũng đưa ra quyết định tối ưu, từ đó giúp AI lựa chọn nước đi mang lại kết quả tốt nhất trong trường hợp xấu nhất có thể xảy ra.

2.2.3. Cài đặt trong dự án

Trong dự án, thuật toán Minimax được hiện thực thông qua hàm **alpha_beta()** trong tệp **minimax.py** với thuật toán Minimax, đồng thời được mở rộng bằng kỹ thuật cắt tia Alpha – Beta nhằm giảm số lượng nút cần duyệt trong quá trình tìm kiếm.

Quy trình xử lý của hàm được thực hiện theo các bước sau:

1. Kiểm tra điều kiện dừng của quá trình tìm kiếm. Khi đạt đến độ sâu giới hạn, hệ thống sẽ sử dụng hàm **quiescence_search()** để tiếp tục đánh giá các trạng thái có khả năng phát độ trao đổi quân thay vì dừng ngay lập tức. Trong trường hợp bàn cờ đang ở trạng thái bị chiếu (check), độ sâu tìm kiếm được mở rộng thêm một mức thông qua kỹ thuật Check Extension nhằm tránh bỏ sót các tình huống chiến thuật quan trọng.
2. Nếu ván cờ đã kết thúc, hàm trả về giá trị đánh giá tương ứng. Đối với trạng thái chiếu hết, điểm số được cộng hoặc trừ thêm theo độ sâu còn lại để AI ưu tiên các phương án chiếu hết nhanh hơn hoặc kéo dài thời gian nếu rơi vào thế bất lợi.
3. Sinh danh sách các nước đi hợp lệ và sắp xếp theo mức độ ưu tiên bằng hàm **score_move()**. Những nước đi như chiếu vua, bắt quân hoặc phong cấp được xét trước nhằm tăng hiệu quả của quá trình tìm kiếm.

```

legal_moves = list(board.legal_moves)
legal_moves.sort(key=lambda m: score_move(board, m), reverse=True)

if is_maximizing:
    max_eval = -float('inf')

    for move in legal_moves:
        board.push(move)
        evaluation = alpha_beta(board, depth - 1, alpha, beta, False, history_set)
        board.pop()
        max_eval = max(max_eval, evaluation)
        alpha = max(alpha, max_eval)

        if beta <= alpha: break

    return max_eval

```

Hình 2.3 Cơ chế sắp xếp nước đi Move Ordering trong hàm `alpha_beta()`

Với nút **Max**, thuật toán lần lượt duyệt các nước đi và lựa chọn giá trị đánh giá lớn nhất. Với nút **Min**, thuật toán lựa chọn giá trị nhỏ nhất theo nguyên lý của Minimax. Trong quá trình duyệt cây tìm kiếm, hệ thống sử dụng phương thức **board.push(move)** để thực hiện nước đi và **board.pop()** để hoàn tác trạng thái trước đó. Cách tiếp cận này cho phép tái sử dụng cùng một đối tượng Board của thư viện **python-chess**, tránh việc tạo nhiều bản sao bàn cờ tại mỗi nút của cây tìm kiếm, từ đó giúp giảm chi phí bộ nhớ và nâng cao hiệu năng thực thi.

2.2.4. Đặc biệt: Ưu tiên chiếu hết nhanh

Trong quá trình cài đặt, hệ thống không sử dụng giá trị cố định cho trạng thái chiếu hết mà cộng (hoặc trừ) thêm giá trị `depth` vào điểm đánh giá:

```
Return -1000 - depth if board.turn == chess.WHITE else 100000 + depth
```

Cách tính này giúp AI không chỉ tìm được phương án chiếu hết mà còn ưu tiên những đường dẫn đưa đến chiến thắng trong số nước đi ít nhất.

Ví dụ:

Trường hợp	Giá trị
Chiếu hết sau 2 nước	100002
Chiếu hết sau 3 nước	100001

Do $100002 > 100001$, AI sẽ ưu tiên phương án chiếu hết sau 2 nước thay vì sau 3 nước.

Ngược lại, khi rơi vào trạng thái bất lợi, điểm đánh giá sẽ được tính theo dạng **-100000 - depth**. Điều này khiến AI có xu hướng lựa chọn những phương án giúp kéo dài thời gian bị chiếu hết, qua đó tạo thêm cơ hội để đối thủ mắc sai lầm trong quá trình thi đấu.

Nhờ cơ chế trên, hệ thống không chỉ tìm được trạng thái thắng hoặc thua mà còn tối ưu hóa số lượng nước đi để đạt được kết quả mong muốn.

2.3. Cắt tỉa Alpha-Beta

2.3.1. Vấn đề của Minimax thuần túy

Thuật toán Minimax truyền thống phải duyệt toàn bộ cây trò chơi đến độ sâu tìm kiếm đã xác định để đánh giá và lựa chọn nước đi tối ưu. Khi số lượng nước đi hợp lệ tại mỗi trạng thái lớn, tổng số nút cần duyệt tăng theo cấp số nhân, dẫn đến chi phí tính toán rất cao.

Để giảm số lượng trạng thái cần đánh giá, kỹ thuật **Alpha-Beta Pruning** được sử dụng nhằm loại bỏ các nhánh chắc chắn không thể ảnh hưởng đến quyết định cuối cùng của thuật toán mà vẫn đảm bảo kết quả lựa chọn giống với Minimax thuần túy.

Thuật toán duy trì đồng thời hai giá trị:

- **Alpha (α):** giá trị tốt nhất mà người chơi **Max (AI)** đã đảm bảo đạt được trên đường đi hiện tại. Ban đầu α được khởi tạo bằng giá trị âm vô cùng ($-\infty$).
- **Beta (β):** giá trị tốt nhất mà người chơi **Min (đối thủ)** đã đảm bảo đạt được trên đường đi hiện tại. Ban đầu β được khởi tạo bằng giá trị dương vô cùng ($+\infty$).

Trong quá trình duyệt cây:

- Tại nút **Max**, giá trị α sẽ được cập nhật nếu tìm thấy phương án tốt hơn.
- Tại nút **Min**, giá trị β sẽ được cập nhật nếu tìm thấy phương án nhỏ hơn.

Khi xuất hiện điều kiện: $\beta \leq \alpha$

thì toàn bộ các nhánh con còn lại của nút hiện tại có thể được loại bỏ (prune) vì chúng không còn khả năng làm thay đổi kết quả lựa chọn cuối cùng của thuật toán.

2.3.2. Minh họa bằng ví dụ

```
a=-inf, b=+inf
+----- MAX (BOT) -----+
|                               |
+-- Nhanh A -----+         |
|   MIN: a=-inf,b=+inf       |
|   La +5 -> b=5              |
|   La +3 -> b=3              |
|   MIN tra ve: 3 -> a=3      |
```



```

|
+-- Nhanh B -----+
|   MIN: a=3, b=+inf
|   La +2 -> b=2
|   b(2) <= a(3) -> CAT! x
|   (Bo qua phan con lai)
+--> BOT chon Nhanh A (=3) --+

```

Hình 2.4: Minh họa cắt tỉa Alpha-Beta - beta-cutoff tại Nhánh B

Trong ví dụ trên, sau khi đánh giá xong nhánh A, thuật toán cập nhật giá trị $\alpha = 3$ tại nút MAX. Khi chuyển sang nhánh B, nút MIN nhận giá trị đầu tiên là 2 và cập nhật $\beta = 2$. Do điều kiện $\beta \leq \alpha$ ($2 \leq 3$) được thỏa mãn, các nút còn lại của nhánh B sẽ bị cắt tỉa mà không cần đánh giá. Việc cắt tỉa này không làm thay đổi kết quả cuối cùng của thuật toán nhưng giúp giảm đáng kể số lượng nút phải duyệt và thời gian thực thi.

2.3.3. Cài đặt trong dự án

Kỹ thuật Alpha – Beta Pruning được cài đặt trực tiếp trong hàm **alpha_beta()** của file **minimax.py**. Sau mỗi lần đánh giá một nút con, thuật toán cập nhật giá trị alpha hoặc beta tương ứng với nút Maximizing và Minimizing. Khi điều kiện **beta <= alpha** được thỏa mãn, vòng lặp sẽ dừng (break) và các nhánh còn lại không tiếp tục được duyệt.

Nhờ cơ chế này, hệ thống loại bỏ được nhiều trạng thái không cần thiết trong quá trình tìm kiếm nhưng vẫn đảm bảo kết quả lựa chọn giống với thuật toán Minimax đầy đủ.

```

if is_maximizing:
    max_eval = -float('inf')

    for move in legal_moves:
        board.push(move)
        evaluation = alpha_beta(board, depth - 1, alpha, beta, False, history_set)
        board.pop()
        max_eval = max(max_eval, evaluation)
        alpha = max(alpha, max_eval)

        if beta <= alpha: break

    return max_eval

else:
    min_eval = float('inf')
    for move in legal_moves:
        board.push(move)
        evaluation = alpha_beta(board, depth - 1, alpha, beta, True, history_set)
        board.pop()
        min_eval = min(min_eval, evaluation)
        beta = min(beta, min_eval)
        if beta <= alpha: break

    return min_eval

```

Hình 2.5: Cài đặt cắt tỉa Alpha-Beta trong minimax.py

2.3.4. Phân tích hiệu quả:

Về mặt lý thuyết, thuật toán Alpha–Beta Pruning có thể giảm đáng kể số lượng nút cần duyệt so với Minimax thuần túy khi các nước đi được xét theo thứ tự thuận lợi. Trong trường hợp lý tưởng, độ phức tạp có thể giảm từ $O(b^d)$ xuống $O(b^{d/2})$

Trường hợp	Độ phức tạp
Minimax thuần túy	$O(b^d)$
Alpha–Beta (trường hợp tốt nhất)	$O(b^{d/2})$
Alpha–Beta (trường hợp xấu nhất)	$O(b^d)$

Bảng 2.2. So sánh độ phức tạp lý thuyết giữa Minimax và Alpha–Beta.

Hiệu quả của Alpha–Beta được cải thiện thêm nhờ cơ chế **Move Ordering**, khi danh sách nước đi được sắp xếp trước bằng hàm `score_move()` để ưu tiên các nước đi có khả năng mang lại lợi thế cao như chiếu vua, bắt quân hoặc phong cấp. Việc kết hợp Alpha–Beta với Move Ordering đã giúp giảm đáng kể khối lượng tìm kiếm và cải thiện tốc độ phản hồi của AI so với việc duyệt toàn bộ cây trò chơi.

2.4. Tìm kiếm Quiescence (Quiescence Search)

2.4.1. Hiệu ứng chân trời (Horizon Effect)

Hiệu ứng chân trời (Horizon Effect) là hiện tượng thuật toán Minimax đánh giá trạng thái bàn cờ tại giới hạn độ sâu tìm kiếm trong khi chuỗi trao đổi quân hoặc các nước đi chiến thuật quan trọng vẫn chưa kết thúc. Điều này có thể khiến AI đưa ra quyết định thiếu chính xác do chỉ quan sát được một phần diễn biến của ván cờ.

Ví dụ, tại nút lá ($\text{depth} = 0$), AI nhận thấy quân Tốt vừa bắt được quân Mã của đối phương nên hàm `evaluate_board()` trả về giá trị tích cực và đánh giá đây là một nước đi có lợi. Tuy nhiên, ngay sau đó đối thủ có thể thực hiện chuỗi bắt quân liên tiếp để giành lại ưu thế. Vì quá trình tìm kiếm đã dừng tại độ sâu giới hạn nên Minimax không quan sát được diễn biến tiếp theo và dẫn đến đánh giá sai.

Ví dụ về Horizon Effect

Tình huống: Tốt vừa ăn quân Mã tại nút lá ($\text{depth} = 0$)

`evaluate_board()` = +325 điểm

→ BOT nghĩ đây là nước đi có lợi và chọn

NHUNG: Ngay sau đó, đối phương ăn lại quân Tốt (-100), đối phương tiếp tục đổi quân và giành lợi thế

Kết quả thực tế: thấp hơn nhiều so vs đánh giá ban đầu

Minimax đã dừng tìm kiếm tại $\text{depth} = 0$ nên không quan sát được chuỗi trao đổi quân tiếp theo

2.4.2. Giải pháp Quiescence Search

Để khắc phục Horizon Effect, ta có kỹ thuật Quiescence Search. Khi độ sâu tìm kiếm đạt đến 0 và bàn cờ không ở trạng thái bị chiếu, thuật toán không đánh giá ngay bằng `evaluate_board()` mà tiếp tục mở rộng việc tìm kiếm đối với các nước ăn quân (capture moves).

Quá trình này chỉ kết thúc khi không còn nước ăn quân hợp lệ, giúp trạng thái được đánh giá ổn định hơn trước khi trả về giá trị cho thuật toán Minimax.

Nhờ đó, AI hạn chế được các trường hợp đánh giá quá lạc quan hoặc quá bi quan tại các nút lá khi trên bàn cờ vẫn còn các chuỗi trao đổi quân quan trọng.

depth=0

Minimax thông thường: `evaluate()` = +900 → trả về ngay

Quiescence Search:

`stand_pat = evaluate()` = +900

Tiếp tục xét các nước:

-> Đối phương ăn lại hậu: `evaluate()` = -800

-> AI tiếp tục ăn lại: `evaluate()` = +200

-> Không còn nước bắt quân -> dừng

Trả về: +200 <- CHÍNH XÁC HƠN

Hình 2.6: *Quiescence Search* ngăn đánh giá sai tại nút lá

2.4.3 Cài đặt trong dự án

Trong dự án, khi thuật toán chuyển sang `quiescence_search()`, trạng thái bàn cờ trước tiên được đánh giá thông qua biến `stand_pat` bằng cách gọi hàm `evaluate_board()`. Giá trị này được sử dụng để cập nhật ngưỡng `alpha` hoặc `beta` trước khi tiếp tục mở rộng cây tìm kiếm.

Sau đó, hệ thống sử dụng `board.generate_legal_captures()` để sinh danh sách các nước ăn quân hợp lệ và sắp xếp bằng hàm `score_move()` nhằm ưu tiên những nước đi có giá trị cao hơn. Mỗi nước đi được thực hiện bằng `board.push()`, đánh giá đệ quy thông qua `quiescence_search()`, rồi hoàn tác bằng `board.pop()` để tiếp tục duyệt các nhánh khác.

Nhờ cơ chế này, AI có thể tiếp tục xử lý các chuỗi trao đổi quân quan trọng ngay cả khi đã đạt đến giới hạn độ sâu tìm kiếm, từ đó giảm ảnh hưởng của Horizon Effect và nâng cao độ chính xác của quá trình đánh giá.

```
def quiescence_search(board, alpha, beta, is_maximizing):
    stand_pat = evaluate_board(board)
    if is_maximizing:
        if stand_pat >= beta: return beta
        if alpha < stand_pat: alpha = stand_pat
        capture_moves = list(board.generate_legal_captures())
        capture_moves.sort(key=lambda m: score_move(board, m), reverse=True)
        for move in capture_moves:
            board.push(move)
            score = quiescence_search(board, alpha, beta, False)
            board.pop()
            if score >= beta: return beta
            if score > alpha: alpha = score
    return alpha
```

Hình 2.7: Cài đặt hàm *quiescence_search()* trong *minimax.py*

2.5. Hàm đánh giá tĩnh bàn cờ

2.5.1. Giá trị vật chất (Material Value)

Trong hàm *evaluate_board()*, thành phần đầu tiên được sử dụng để đánh giá bàn cờ là giá trị vật chất (Material Value). Mỗi loại quân được gán một trọng số cố định thông qua biến *PIECE_VALUES*, phản ánh tầm quan trọng tương đối của quân cờ trong ván đấu.

Quân cờ	Ký hiệu	Giá trị (centipawn)	Ghi chú
Tốt (Pawn)	P/p	100	Đơn vị cơ bản (1 pawn = 1.00)
Mã (Knight)	N/n	325	Mạnh ở trung cuộc
Tượng (Bishop)	B/b	330	Tương đối rất mạnh
Xe (Rook)	R/r	500	Mạnh ở tàn cuộc
Hậu (Queen)	Q/q	900	Quân mạnh nhất
Vua (King)	K/k	20000	Không thể ăn / Giá trị tượng trưng

Bảng 2.3. Giá trị vật chất được sử dụng trong *PIECE_VALUES* của *minimax.py*

2.5.2. Piece-Square Tables

Bên cạnh giá trị vật chất, dự án sử dụng Piece-Square Table (PST) để đánh giá mức độ thuận lợi của vị trí từng quân cờ trên bàn cờ. Mỗi ô trong bảng tương ứng với một giá trị thưởng hoặc phạt, giúp AI ưu tiên đưa quân đến các vị trí có lợi về mặt chiến thuật.

```
PAWN_TABLE = [
    0, 0, 0, 0, 0, 0, 0, 0,
    50, 50, 50, 50, 50, 50, 50, 50,
    10, 10, 20, 30, 30, 20, 10, 10,
    5, 5, 10, 25, 25, 10, 5, 5,
    0, 0, 0, 20, 20, 0, 0, 0,
    5, -5, -10, 0, 0, -10, -5, 5,
    5, 10, 10, -20, -20, 10, 10, 5,
    0, 0, 0, 0, 0, 0, 0, 0
]
```

Hình 2.8. Piece-Square Table áp dụng cho quân Tốt trong *minimax.py*

Trong quá trình tính điểm, cùng một bảng PST được sử dụng cho cả hai bên. Đối với quân Trắng, chỉ số ô cờ được chuyển đổi bằng **chess.square_mirror()** để đảm bảo việc đánh giá diễn ra theo cùng một hướng với quân Đen. Cách xử lý này giúp tái sử dụng chung một bảng dữ liệu và đơn giản hóa quá trình cài đặt

2.5.3. Các thành phần đánh giá bổ sung

Bên cạnh giá trị vật chất và Piece-Square Table, hàm `evaluate_board()` còn bổ sung một số tiêu chí heuristic nhằm phản ánh chính xác hơn chất lượng của trạng thái bàn cờ.

Trong dự án, ván cờ được xem là bước vào giai đoạn tàn cuộc khi tổng số Hậu và Xe còn lại trên bàn cờ không vượt quá 2 quân. Khi đó:

- KING_MIDDLE_TABLE được thay bằng KING_END_TABLE để khuyến khích Vua di chuyển vào trung tâm bàn cờ.
- Hệ số nhân của Piece-Square Table được tăng từ 0.6 lên 1.0, giúp vị trí quân cờ có ảnh hưởng lớn hơn đến điểm đánh giá.

Ngoài ra, hàm đánh giá còn bổ sung các heuristic khác:

- Thưởng cho bên vẫn còn quyền nhập thành (**castling rights**) và phạt trường hợp Vua đã di chuyển nhưng chưa nhập thành.
- Thưởng **35 điểm** khi sở hữu cặp Tượng (bishop pair).
- Cộng hoặc trừ điểm theo số lượng nước đi hợp lệ (mobility), với trọng số 2 điểm cho mỗi nước đi.

Các thành phần này giúp AI đánh giá không chỉ dựa trên số lượng quân cờ mà còn xét đến độ an toàn của Vua, khả năng kiểm soát thế trận và tính cơ động của các quân cờ.

```
def is_endgame(board):
    heavy_pieces = len(board.pieces(chess.QUEEN, chess.WHITE)) + \
len(board.pieces(chess.QUEEN, chess.BLACK)) + \
len(board.pieces(chess.ROOK, chess.WHITE)) + \
len(board.pieces(chess.ROOK, chess.BLACK))
    return heavy_pieces <= 2
```

Hình 2.9 Phát hiện giai đoạn tàn cuộc (Endgame Detection)

2.6. Sắp xếp nước đi (Move Ordering)

2.6.1. Nguyên lý

Hiệu quả của thuật toán Alpha-Beta phụ thuộc rất lớn vào thứ tự duyệt các nước đi. Nếu những nước đi mạnh được xem xét trước, điều kiện cắt tia ($\beta \leq \alpha$) sẽ xuất hiện sớm hơn, từ đó giảm đáng kể số lượng nhánh cần duyệt.

Trong dự án, trước khi tìm kiếm, danh sách các nước đi hợp lệ được sắp xếp theo giá trị trả về của hàm `score_move()`:

```
Legalmoves = list(board.legal_moves)

Legalmove.sort(key=lambda m: scoremove(board, m), reverse = True)
```

Hình 2.10. Sắp xếp danh sách nước đi theo giá trị `score_move()`

2.6.2. Hàm đánh giá thứ tự nước đi

Hàm `score_move()` gán điểm ưu tiên cho từng nước đi dựa trên một số heuristic đơn giản nhằm đưa các nước đi quan trọng lên đầu danh sách.

Loại nước đi	Điểm ưu tiên	Ý nghĩa
Phong cấp tốt	+9000	Ưu tiên tạo Hậu hoặc quân mạnh hơn
Chiế vua (gives_check)	+5000	Ép đối phương phản ứng

Ăn quân (is_capture)	$1000 \times \text{giá trị quân bị ăn} - \text{giá trị quân bị tấn công}$	Ưu tiên đổi quân có lợi
Nước đi thường	0	Không có điểm thưởng

Bảng 2.4 Thứ tự ưu tiên hàm *score_move()*

Đối với các nước ăn quân, dự án sử dụng nguyên tắc **MVV-LVA (Most Valuable Victim – Least Valuable Attacker)**. Theo đó, hệ thống ưu tiên những nước ăn quân có giá trị lớn bằng quân có giá trị nhỏ.

Ví dụ:

- Tốt (100) ăn Hậu (900): $1000 \times 900 - 100 = 899900$
- Hậu (900) ăn Tốt (100): $1000 \times 100 - 900 = 99100$

Do đó, nước **Tốt ăn Hậu** sẽ được xét trước vì mang lại lợi thế vật chất lớn hơn.

2.6.3. Vai trò trong dự án

Move Ordering **không làm thay đổi kết quả cuối cùng của thuật toán Minimax**, mà chỉ thay đổi **thứ tự duyệt các nhánh**. Khi các nước đi mạnh được xét trước, Alpha-Beta có thể cắt tỉa nhiều nhánh hơn, từ đó giảm thời gian tìm kiếm và tăng tốc độ phản hồi của chương trình.

Trong dự án, cơ chế này được áp dụng đồng thời tại:

- Hàm `find_best_move()`, trước khi bắt đầu tìm kiếm.
- Hàm `alpha_beta()`, tại mỗi nút của cây tìm kiếm.
- Hàm `quiescence_search()`, khi sắp xếp các nước ăn quân.

CHƯƠNG III: THIẾT KẾ HỆ THỐNG

3.1.Kiến trúc hệ thống

Titan Chess AI được thiết kế theo mô hình phân tầng rõ ràng, tách biệt giữa lớp giao diện (Presentation Layer), lớp điều khiển logic (Control Layer), và lớp AI (AI Engine Layer). Kiến trúc này đảm bảo tính module hóa cao, dễ bảo trì và mở rộng.

Các module:

main.py (GameController): Điều khiển luồng game, quản lý trạng thái, xử lý sự kiện người dùng, lưu PGN

GUI.py (ChessGUI): Vẽ bàn cờ và quân cờ, xử lý click chuột, hiển thị lịch sử nước đi

minimax.py: Thuật toán AI: Minimax, Alpha-Beta Pruning, Quiescence Search, hàm đánh giá

chess (thư viện): Quản lý luật cờ vua: nước đi hợp lệ, chiếu hết, hòa cờ, PGN/FEN

main.spec (PyInstaller): Cấu hình đóng gói thành file .exe độc lập

3.2.Luồng xử lý chính

Người chơi click chọn quân và click ô đích trên giao diện ChessGUI.

↓

handle_player_move() kiểm tra tính hợp lệ của nước đi, cập nhật trạng thái bàn cờ, ghi lịch sử SAN.

↓

Nếu ván cờ chưa kết thúc, trigger_ai_thread() khởi động một luồng (thread) mới để AI tính toán nước đi — đảm bảo giao diện không bị đóng băng (UI freeze).

↓

async_ai_task() trong luồng phụ gọi find_best_move() trong minimax.py, trả về nước đi tốt nhất.

↓

`apply_ai_move()` được gọi trên luồng chính (thông qua `root.after()`) để cập nhật bàn cờ an toàn với Tkinter.



Kiểm tra kết thúc ván cờ sau mỗi nước đi. Nếu kết thúc, lưu PGN và hiển thị popup kết quả

3.3.Module GameController (main.py)

3.3.1.Khởi tạo và cấu hình

GameController là lớp trung tâm điều phối toàn bộ ứng dụng. Khi khởi tạo, đối tượng này thiết lập:

Cửa sổ Tkinter chính (root) với tiêu đề và kích thước cố định 830x620 pixel.

Trạng thái game ban đầu: bàn cờ mới (`chess.Board()`), lịch sử nước đi rỗng, độ sâu `AI_DEPTH = 4`.

Thông tin người chơi: tên, ELO (mặc định 1500 cho người chơi và 2000 cho bot).

Cờ trạng thái: `player_resigned`, `ai_is_thinking` để đồng bộ hóa giữa các luồng.

3.3.2.Xử lý đa luồng

Một trong những thách thức kỹ thuật quan trọng nhất trong dự án là đảm bảo giao diện người dùng không bị đóng băng trong khi AI đang tính toán. Với độ sâu `depth=4` và Quiescence Search, AI có thể mất vài giây để tìm nước đi tốt nhất.

Giải pháp được áp dụng là Python threading:

`trigger_ai_thread()` tạo một thread daemon mới, chạy `async_ai_task()` trong nền.

Sau khi tính toán xong, kết quả được gửi về luồng chính bằng `root.after(0, apply_ai_move, ai_move)` — đây là cách an toàn duy nhất để cập nhật Tkinter GUI từ luồng phụ.

Biến `ai_is_thinking = True` ngăn người dùng đi nước trong khi AI đang suy nghĩ.

3.3.3. Kiểm tra trạng thái kết thúc

Phương thức `check_is_draw_active()` kiểm tra toàn diện các điều kiện hòa cờ theo luật FIDE:

Stalemate: bên đến lượt không có nước đi hợp lệ nhưng vua chưa bị chiếu.

Insufficient material: cả hai bên không đủ lực lượng để chiếu hết (ví dụ vua vs. vua).

Threefold repetition: thế cờ lặp lại 3 lần (dùng `can_claim_threefold_repetition()`).

Fifty-move rule: 50 nước đi liên tiếp không có ăn quân và không đi Tốt.

3.4. Module giao diện (GUI.py)

Module ChessGUI được import từ GUI.py (không có trong mã nguồn đính kèm nhưng được khai báo trong main.py). Dựa trên cách sử dụng trong GameController, có thể xác định các thuộc tính và phương thức chính:

`is_player_turn`: Cờ boolean kiểm soát xem người chơi có thể di chuyển quân không

`selected_square`: Ô cờ đang được chọn (None nếu chưa chọn)

`draw()`: Vẽ lại toàn bộ bàn cờ, quân cờ và các ô highlight

`update_move_log(san_list)`: Cập nhật panel hiển thị lịch sử nước đi theo ký hiệu SAN

3.5. Tích hợp thư viện chess (python-chess)

Thư viện python-chess của tác giả Niklas Fiekas là nền tảng quản lý luật cờ vua cho Titan Chess AI. Thư viện cung cấp:

`chess.Board`: đối tượng quản lý trạng thái bàn cờ, tự động tạo tất cả nước đi hợp lệ.

`board.legal_moves`: iterator tất cả nước đi hợp lệ cho bên đến lượt.

`board.generate_legal_captures()`: chỉ sinh ra các nước ăn quân — dùng cho Quiescence Search.

`board.san(move)`: chuyển nước đi sang ký hiệu SAN chuẩn (Standard Algebraic Notation).

`board.is_checkmate()`, `board.is_stalemate()`, `board.is_game_over()`: kiểm tra trạng thái kết thúc.

`chess.pgn`: hỗ trợ đọc/ghi file PGN chuẩn FIDE.

`chess.square_mirror(square)`: đảo vị trí ô cờ — thiết yếu để áp dụng PST đúng cho cả hai màu quân.

CHƯƠNG IV. THUẬT TOÁN AI - PHÂN TÍCH CHI TIẾT

4.1. Cấu Trúc Hàm `find_best_move()`

Hàm `find_best_move()` là điểm vào chính của engine AI, nhận các tham số:

`board`: bản sao của bàn cờ hiện tại (đã được copy trong `async_ai_task()` để thread-safe).

`depth`: độ sâu tìm kiếm (mặc định = 4 trong cài đặt hiện tại).

`history`: danh sách FEN các trạng thái đã qua, dùng để phát hiện lặp thế.

`ai_color`: màu quân AI đang chơi (`chess.WHITE` hoặc `chess.BLACK`).

Thuật toán phân nhánh theo màu quân AI. Khi AI chơi trắng (Maximizing), hàm tìm nước đi có `board_value` cao nhất. Khi AI chơi đen (Minimizing), tìm nước đi có `board_value` thấp nhất. Điều này đảm bảo tính nhất quán bất kể AI chơi phe nào.

Một đặc điểm đáng chú ý: nếu AI đang chơi trắng và đây là nước đi đầu tiên (`fullmove_number == 1`), thay vì chạy Minimax, AI chọn ngẫu nhiên một trong các nước khai cục cổ điển: e4, d4, Nf3, c4. Điều này tạo ra sự đa dạng và giảm thời gian tính toán không cần thiết ở khai cục.

4.2. Phân Tích Hàm `evaluate_board()`

4.2.1 Xử Lý Trạng Thái Kết Thúc

Trước khi tính điểm, hàm kiểm tra các trạng thái đặc biệt:

Chiếu hết (checkmate): trả về -99999 nếu bên trắng bị chiếu hết, +99999 nếu bên đen bị chiếu hết. Giá trị tuyệt đối này đảm bảo AI luôn ưu tiên nước chiếu hết hơn bất kỳ lợi thế vật chất nào.

Hòa (stalemate, insufficient material, fifty moves): trả về 0. Điều này quan trọng để AI tránh sa vào thế hòa khi đang có lợi thế lớn.

4.2.2 Xử Lý Đúng Hướng PST Cho Hai Màu Quân

Một vấn đề tinh tế trong triển khai PST là các bảng điểm được thiết kế từ góc nhìn của quân trắng (hàng 1 ở dưới). Khi áp dụng cho quân đen (hàng 8 ở dưới), cần lật ngược chiều bảng. Titan Chess AI xử lý điều này bằng `chess.square_mirror()`:

```
# Quân Trắng: Đọc chỉ số xuôi theo ma trận gốc
for square in board.pieces(piece_type, chess.WHITE):
    score += PIECE_TABLES[piece_type][square]

# Quân Đen: Đọc ngược hướng, phải dùng square_mirror
for square in board.pieces(piece_type, chess.BLACK):
    b_idx = chess.square_mirror(square) # Đảo ngược vị trí
    score -= PIECE_TABLES[piece_type][b_idx]
```

Hàm `chess.square_mirror(square)` trả về ô đối xứng qua trục ngang (ví dụ `a1` ↔ `a8`, `e4` ↔ `e5`). Đây là chi tiết kỹ thuật quan trọng — sai sót ở đây sẽ khiến AI đánh giá sai vị trí quân đen, dẫn đến nước đi kém chất lượng.

4.2.3 Phát Hiện Giai Đoạn Tàn Cuộc

Hàm `is_endgame()` kiểm tra giai đoạn ván cờ dựa trên số lượng quân nặng còn lại (Hậu và Xe). Khi tổng số Hậu và Xe trên bàn cờ ≤ 2 , ván cờ được coi là tàn cuộc.

Trong tàn cuộc, hai thay đổi quan trọng được áp dụng:

Bảng vị trí Vua chuyển từ `KING_MIDDLE_TABLE` sang `KING_END_TABLE`: Vua cần tiến ra trung tâm để tham gia chiến đấu trong tàn cuộc.

Trọng số PST được điều chỉnh: Trong khai cục/trung cuộc, PST được nhân hệ số 0.6 (giảm tầm quan trọng vị trí để ưu tiên phát triển quân). Trong tàn cuộc, hệ số là 1.0 (vị trí quan trọng tối đa)

4.3. Chi Tiết Triển Khai Quiescence Search

Quiescence Search trong Titan Chess AI được triển khai theo thuật toán chuẩn với điểm đánh giá tĩnh (`stand_pat`) làm điểm khởi đầu:

1. Tính `stand_pat = evaluate_board(board)` — điểm đánh giá nếu dừng tìm kiếm tại đây.
2. Nếu `stand_pat >= beta` (cắt tỉa beta): trả về beta ngay lập tức.
3. Cập nhật `alpha = max(alpha, stand_pat)`.
4. Duyệt tất cả nước ăn quân (`generate_legal_captures()`), đã được sắp xếp theo MVV-LVA.
5. Với mỗi nước ăn quân, đệ quy tiếp tục Quiescence Search với đảo ngược `is_maximizing`.

6. Áp dụng cắt tỉa alpha-beta cho từng nước.

Điểm mấu chốt là bước (1): nếu thế cờ hiện tại đã tốt hơn beta, ta dừng ngay — vì ngay cả khi không ăn thêm quân nào, thế cờ đã đủ tốt để loại bỏ nhánh này. Đây gọi là 'standing pat' — giữ nguyên mà không cần ăn thêm.

4.4. Phân Tích Độ Phức Tạp và Hiệu Suất

Với depth=4 và branching factor trung bình ~35 của cờ vua, số nút lý thuyết cần duyệt:

Minimax thuần: $35^4 = 1,500,625$ nút

Alpha-Beta tệ nhất: vẫn $35^4 = 1,500,625$ nút

Alpha-Beta trung bình: khoảng $35^{2.5} \approx 216,000$ nút

Alpha-Beta tốt nhất (nước đi được sắp xếp hoàn hảo): $35^2 = 1,225$ nút

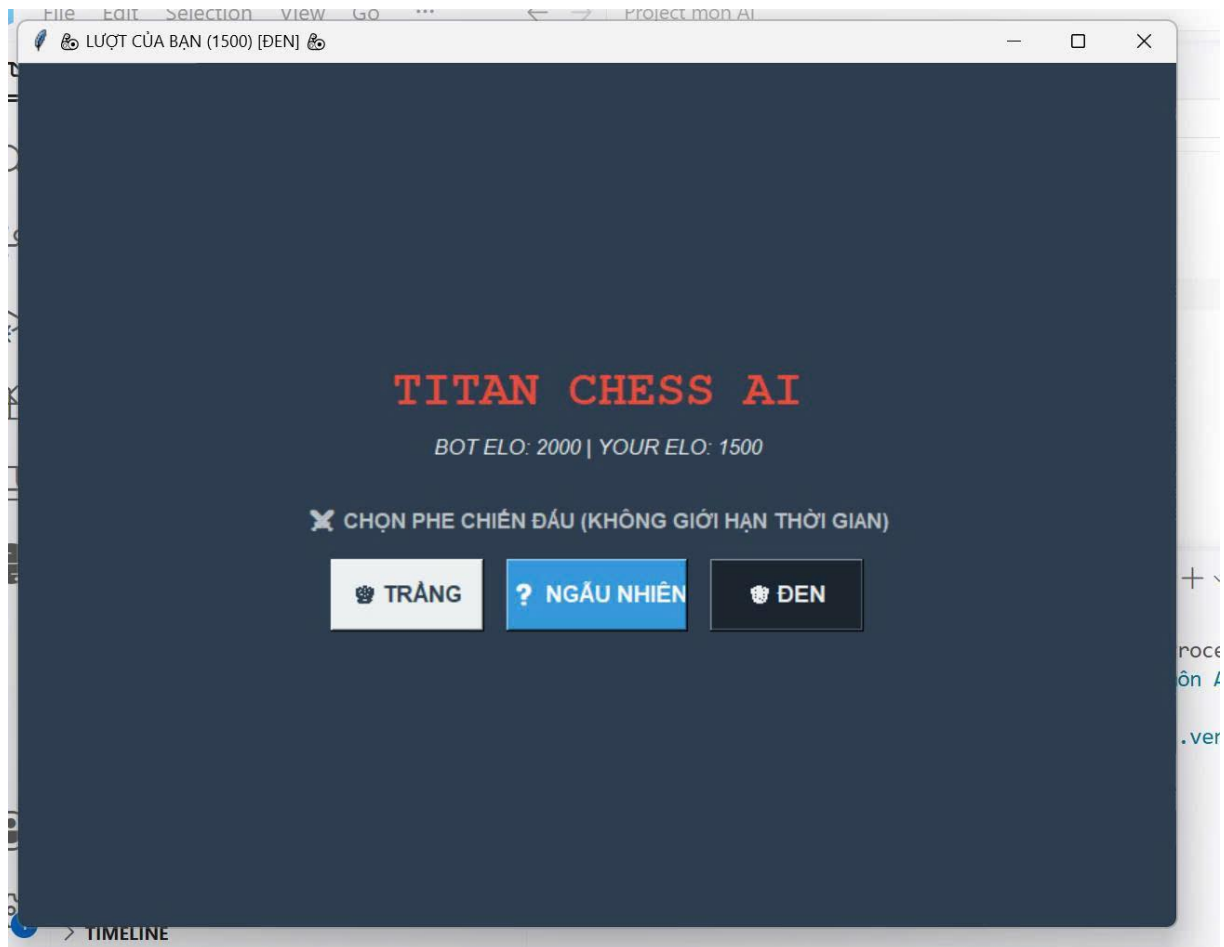
Nhờ move ordering và Alpha-Beta Pruning, Titan Chess AI thực tế chỉ cần duyệt khoảng 50,000-200,000 nút mỗi lượt, cho phép tính toán xong trong vài giây trên phần cứng hiện đại.

CHƯƠNG V. GIAO DIỆN NGƯỜI DÙNG

5.1. Thiết Kế Giao Diện

Giao diện người dùng được xây dựng bằng Tkinter — thư viện GUI tiêu chuẩn đi kèm Python. Toàn bộ giao diện được thiết kế theo theme tối (dark theme) với màu nền #2c3e50 (xanh đen), tạo cảm giác chuyên nghiệp và giảm mỏi mắt khi chơi lâu.

Kích thước cửa sổ chính là 830x620 pixel — được tính toán cẩn thận để bàn cờ 8x8 (640x640 pixel với ô 80px) vừa vặn với panel thông tin bên phải.



Hình 5.1. Giao diện chính

5.2. Màn Hình Chọn Phe (Start Menu)

Khi khởi động, người dùng thấy màn hình chọn phe với thiết kế gọn gàng, hiển thị tên bot (TITAN CHESS AI) và ELO của cả hai bên. Có ba lựa chọn:

♔ TRẮNG: Người chơi đi trước (quân trắng), bàn cờ hiển thị với hàng trắng ở dưới.

? NGẪU NHIÊN: Chọn ngẫu nhiên một trong hai màu.

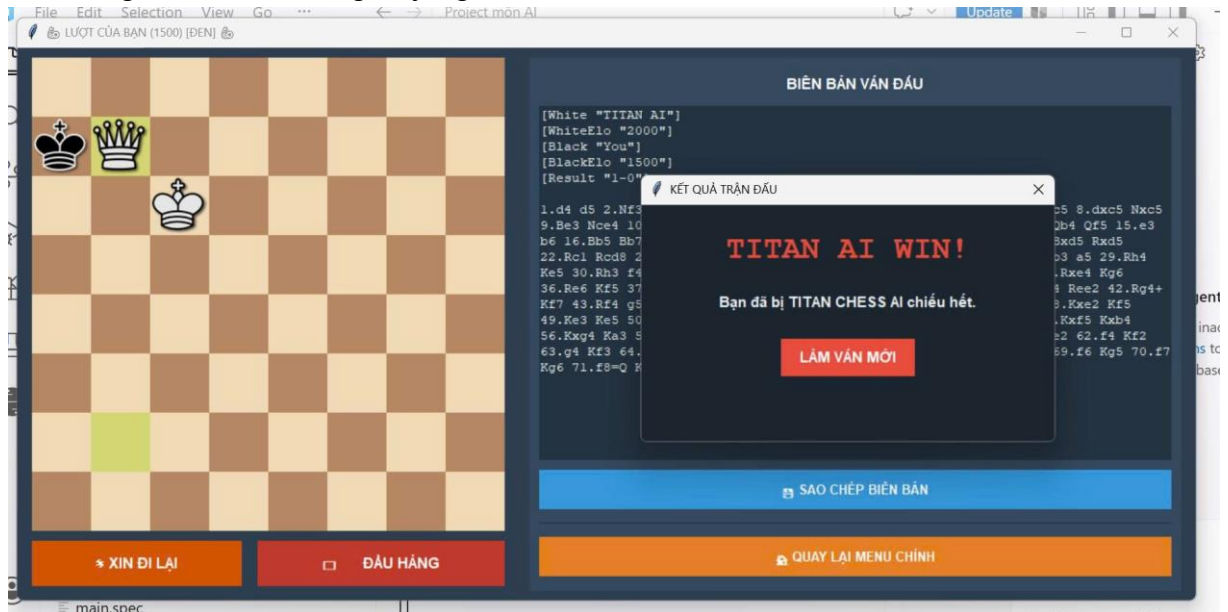
♚ ĐEN: Người chơi đi sau, bàn cờ hiển thị với hàng đen ở dưới. AI sẽ thực hiện nước đi đầu tiên ngay khi màn hình chơi được hiển thị.

5.3. Màn Hình Chơi Chính

Màn hình chơi chính bao gồm các thành phần:

1. Bàn cờ 8x8: Các ô cờ xen kẽ màu sáng/tối. Quân cờ được vẽ bằng ký hiệu Unicode chuẩn. Ô được chọn và các nước đi hợp lệ được highlight bằng màu khác nhau.
2. Panel lịch sử nước đi: Hiển thị danh sách nước đi theo ký hiệu SAN (Standard Algebraic Notation), cuộn tự động khi vượt quá chiều cao panel.

3. Nút điều khiển: HOÀN TÁC (undo 2 nước — cả nước AI và nước người chơi), ĐẦU HÀNG, MENU (quay về màn hình chọn phe).
4. Tiêu đề cửa sổ: Cập nhật động theo trạng thái ván cờ — hiển thị lượt đi và thông báo khi AI đang suy nghĩ.



Hình 5.2.Màn hình chơi chính

5.4.Xử Lý Sự Kiện Chuột

Luồng xử lý click chuột theo cơ chế hai bước (two-click selection):

Click lần 1: Chọn ô chứa quân cờ của người chơi. GUI highlight ô được chọn và các ô mà quân đó có thể di chuyển đến (nước đi hợp lệ).

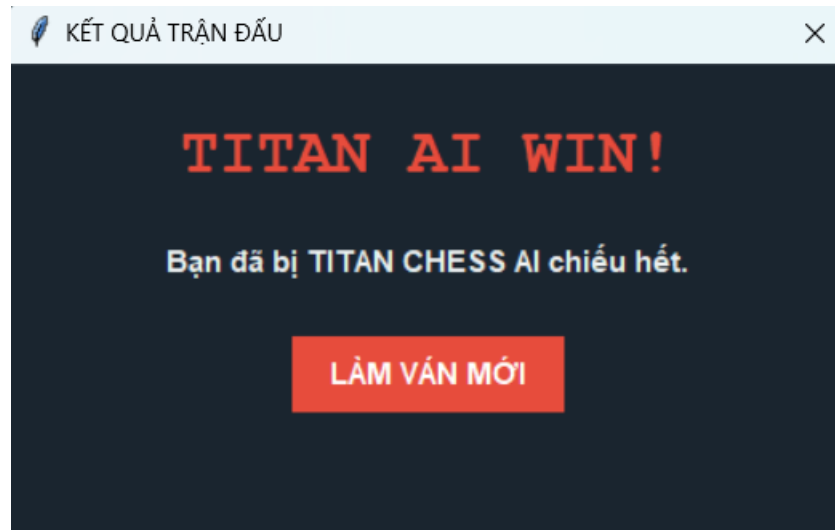
Click lần 2: Click vào ô đích. Nếu ô đích hợp lệ, nước đi được thực thi. Nếu click vào ô không hợp lệ hoặc ô trống, hủy chọn hoặc chọn lại quân khác.

Đặc biệt, khi Tốt đến hàng cuối (rank 7 với trắng, rank 0 với đen), tự động phong hậu (auto-promotion to Queen) — đây là lựa chọn tối ưu trong hầu hết trường hợp. Mặc dù thiếu tùy chọn phong quân khác, nhưng đây là quyết định thiết kế đúng đắn dẫn cho mục tiêu đơn giản hóa UI.

5.5.Popup Kết Quả

Khi ván cờ kết thúc, một cửa sổ popup hiện ra với thông tin kết quả. Popup sử dụng màu sắc ngữ nghĩa:

- Đỏ (#e74c3c): AI thắng.
- Xanh lá (#2ecc71): Người chơi thắng.
- Vàng (#f1c40f): Hòa cờ.



Hình 5.3. Kết quả ván đấu

Popup có nút 'LÀM VÁN MỚI' để bắt đầu ván mới mà không cần khởi động lại ứng dụng. Popup sử dụng `grab_set()` để modal — người dùng không thể tương tác với cửa sổ chính cho đến khi đóng popup.

CHƯƠNG VI. KIỂM THỬ VÀ ĐÁNH GIÁ

6.1. Phương Pháp Kiểm Thử

Titan Chess AI được kiểm thử qua nhiều phương pháp khác nhau để đảm bảo tính đúng đắn của engine cờ vua và chất lượng AI:

6.1.1. Kiểm Thử Luật Cờ Vua

Thư viện `python-chess` đảm bảo 100% tính đúng đắn của luật cờ vua, bao gồm các trường hợp đặc biệt:

Chiếu hết và hòa cờ trong mọi tình huống.

Nhập thành ngắn và nhập thành dài (kingside/queenside castling).

Bắt Tốt qua đường (en passant).

Phong hậu Tốt tại hàng cuối.

Chiếu khám (discovered check) và chiếu đôi (double check).

6.1.2. Kiểm Thử Hàm Đánh Giá

Hàm `evaluate_board()` được kiểm thử thủ công với các thế cờ đặc trưng:

Thế cờ cân bằng (starting position): phải trả về giá trị gần 0.

Thế cờ lợi thế vật chất rõ ràng (ví dụ thêm hậu): phải có giá trị tương ứng $\approx \pm 900$.

Thế cờ chiếu hết: phải trả về ± 99999 .

Thế cờ hòa: phải trả về 0.

6.1.3. Đánh Giá Chất Lượng AI

AI được đánh giá thông qua các ván đấu thử nghiệm với người chơi ở nhiều mức kỹ năng khác nhau, cũng như so sánh nước đi của AI với đề xuất của Stockfish (engine mạnh nhất thế giới) trong các thế cờ tương tự.

6.2. Kết quả kiểm thử

Mục kiểm thử	Kết quả	Ghi chú
Tìm kiếm nước đi bằng Minimax + Alpha-Beta	Đạt	AI lựa chọn được nước đi hợp lệ trong mọi trạng thái kiểm thử
Phát hiện chiếu hết	Đạt	Xử lý chính xác các tình huống kết thúc ván cờ
Phát hiện hòa cờ	Đạt	Hỗ trợ các trạng thái hòa được thư viện python-chess cung cấp
Hiệu năng tìm kiếm (depth = 4)	2–4 giây	Phụ thuộc số lượng nước đi khả dụng
Đa luồng giao diện	Đạt	Giao diện không bị treo khi AI đang tính toán
Lưu ván cờ PGN	Đạt	Xuất được file PGN để lưu và phân tích lại
Hoàn tác nước đi	Đạt	Khôi phục chính xác trạng thái bàn cờ trước đó
Đầu hàng	Đạt	Cập nhật kết quả ván đấu chính xác

6.3.Ước Tính ELO

Mặc dù không có phương pháp chính xác để đo ELO của một engine tự xây dựng mà không tổ chức nhiều ván đấu có kiểm soát, dựa trên các tham số kỹ thuật và kết quả thực tế, Titan Chess AI được ước tính đạt trình độ tương đương người chơi nghiệp dư đến trung cấp. Do chưa thực hiện đánh giá ELO theo quy trình chuẩn hóa, con số ELO chỉ mang tính tham khảo.

Depth=4 với Quiescence Search tương đương nhiều engine cờ vua nghiệp dư ở mức 1400-1700 ELO.

PST và hàm đánh giá nâng cao giúp AI có nước đi hợp lý trong khai cục và trung cuộc.

Tuy nhiên, thiếu các kỹ thuật nâng cao như transposition table, iterative deepening, null-move pruning có thể làm giảm hiệu quả trong một số thế cờ phức tạp.

Ván đấu ví dụ trong latest_game.pgn minh chứng AI có khả năng thực hiện chiến lược nhiều nước: hi sinh thế để mở bàn (nước 6.Nxe5), phát triển tấn công có hệ thống và kết thúc bằng chiếu hết đẹp (16.Qxe8#).

CHƯƠNG VII. KẾT LUẬN, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

7.1.Kết luận

Titan Chess AI là một dự án hoàn chỉnh, thể hiện sự kết hợp thành công giữa lý thuyết Trí Tuệ Nhân Tạo và thực hành lập trình Python. Dự án đã đạt được tất cả các mục tiêu đề ra ban đầu:

Thuật toán AI mạnh mẽ: Minimax với Alpha-Beta Pruning và Quiescence Search hoạt động chính xác, cho phép tìm kiếm đến độ sâu 4 trong thời gian chấp nhận được.

Hàm đánh giá toàn diện: Kết hợp giá trị vật chất, Piece-Square Tables, bonus chiến lược và phát hiện giai đoạn tàn cuộc, tạo ra nước đi có tính chiến lược cao.

Giao diện người dùng thân thiện: Tkinter GUI với đầy đủ tính năng cần thiết, xử lý đa luồng để UI luôn phản hồi.

Chuẩn hóa dữ liệu: Lưu trữ ván đấu theo định dạng PGN quốc tế, tương thích với mọi phần mềm phân tích cờ vua.

Triển khai dễ dàng: Đóng gói thành file .exe độc lập bằng PyInstaller.

7.2.Hạn chế

7.2.1 Hạn Chế Kỹ Thuật AI

Dù mạnh mẽ, Titan Chess AI vẫn còn một số hạn chế kỹ thuật so với các engine hiện đại:

Thiếu Transposition Table (bảng băm lưu trữ kết quả đã tính): Engine hiện tại có thể tính toán lại cùng một thế cờ nhiều lần trong quá trình tìm kiếm. Transposition Table sẽ lưu kết quả đã tính và tra cứu lại, giảm đáng kể thời gian tính toán.

Thiếu Iterative Deepening: Thay vì tìm kiếm thẳng đến depth=4, Iterative Deepening sẽ tìm kiếm depth=1, rồi depth=2, 3, 4 lần lượt. Kết quả tìm kiếm sâu hơn được dùng để cải thiện move ordering cho lần tìm kiếm tiếp theo, tăng đáng kể hiệu quả alpha-beta.

Thiếu Null-Move Pruning: Kỹ thuật cho phép engine 'bỏ lượt' để kiểm tra nhanh xem thế cờ có đủ tốt để cắt tỉa không, giảm khoảng 15-20% nút cần duyệt.

Thiếu Late Move Reduction (LMR): Giảm độ sâu tìm kiếm cho các nước đi ít triển vọng ở cuối danh sách move ordering.

Không có opening book: AI phải tự tính toán từ nước đầu tiên thay vì tra cứu database khai cục định sẵn.

Không có endgame tablebases: Trong tàn cuộc ít quân, engine không thể đảm bảo kết quả hoàn hảo (win/draw/lose) như Syzygy tablebases.

7.2.1.Hạn Chế Giao Diện

Chỉ hỗ trợ phong hậu Tốt thành Hậu (auto-queen), không cho phép chọn phong thành Mã, Tượng hoặc Xe — đôi khi underpromotion là nước đi tốt hơn.

Không có đồng hồ cờ (chess clock) — hiện tại chỉ hỗ trợ chế độ unlimited time.

Không có chức năng phân tích ván cờ (game analysis) sau khi kết thúc.

Không hỗ trợ chơi online hoặc chơi hai người cùng máy.

7.3.Hướng Phát Triển Tương Lai

7.3.1 Cải Tiến Engine AI

Các cải tiến kỹ thuật có thể nâng ELO của engine lên đáng kể:

Triển khai Transposition Table với Zobrist Hashing: Tạo hash 64-bit duy nhất cho mỗi trạng thái bàn cờ, lưu kết quả alpha-beta vào bảng băm. Ước tính tăng hiệu suất 30-50%.

Iterative Deepening với Move Killer và History Heuristic: Cải thiện move ordering bằng cách nhớ các nước đi đã gây cắt tỉa beta trong các tìm kiếm trước.

Principal Variation Search (PVS): Biến thể nâng cao của Alpha-Beta, sử dụng cửa sổ null-window (α , $\alpha+1$) cho các nước không phải PV, tiết kiệm tính toán.

Tích hợp Neural Network (NNUE): File `titan_chess_model_best.pth` trong `main.spec` gợi ý đây là hướng phát triển đã được lên kế hoạch. NNUE (Efficiently Updatable Neural Network) là công nghệ được Stockfish áp dụng thành công từ 2020.

7.3.1.Mở Rộng Tính Năng

Hỗ trợ đồng hồ cờ: Blitz (5 phút), Rapid (15 phút), Classical (90 phút).

Phân tích ván cờ: Sau ván đấu, hiển thị đánh giá từng nước và gợi ý nước tốt hơn.

Opening Book: Tích hợp database khai cục chuẩn (ví dụ Polyglot format).

Hỗ trợ nhập FEN/PGN: Cho phép người dùng nhập thế cờ từ FEN string hoặc load PGN có sẵn.

Giao diện cải tiến: Thêm hiệu ứng animation khi di chuyển quân, âm thanh, theme sáng/tối.

Chế độ chơi đa người (Two-Player): Hỗ trợ chơi giữa hai người trên cùng một máy hoặc qua mạng.

TÀI LIỆU THAM KHẢO

- [1] Chess.com Analysis Board. Công cụ phân tích ván cờ và đánh giá chất lượng nước đi. <https://www.chess.com/analysis>
- [2] Chess Programming Wiki – Simplified Evaluation Function. Tomasz Michniewski. Tài liệu về giá trị quân cờ (Piece Values), bảng điểm vị trí (Piece-Square Tables) và hàm đánh giá trong Chess Engine. https://www.chessprogramming.org/Simplified_Evaluation_Function
- [3] python-chess Documentation. Tài liệu chính thức của thư viện python-chess. <https://python-chess.readthedocs.io>
- [4] GeeksforGeeks. Minimax Algorithm in Game Theory. <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/>
- [5] GeeksforGeeks. Alpha-Beta Pruning for Minimax Algorithm. <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/>
- [6] Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach. Pearson Education.
- [7] Shannon, C. E. (1950). Programming a Computer for Playing Chess. Philosophical Magazine.
- [8] Knuth, D. E., & Moore, R. W. (1975). An Analysis of Alpha-Beta Pruning. Artificial Intelligence Journal.

[9] Maharaj, S., Polson, N., & Turk, A. (2021). Chess AI: Competing Paradigms for Machine Intelligence.

[10] Monroe, D., Eilender, G., Chalmers, P., Tang, Z., & Anderson, A. (2026). Chessformer: A Unified Architecture for Chess Modeling.

